SOA

SOA
- Implementacion soa
- Errores SMS
  - Funcionamiento TRB Entradas:
  - Funcionamiento TRB Salidas:
- Funcionalidades
  - CLIMA
  - Cuando se cambia estado desde la APP:
  - Cuando se pulsa en Pulsador:
    - TH_ pulsador: cosido en el th del pulsador (argumentos json)
  - Cadena de Seguridad:
    - Th cadena_seguridad salida:
- Domo:
  - Triggers domo:
    - Luces:
    - Clima:
    - Iot:
    - Medidores:
    - Historicos:
  - Eventos
    - Medidores:
- Versiones de SOA:
  - libsoav1 (v1) (rudy)
  - libsoav1_unitp (version intermedia)
  - libsoav1_unitpbd: (v1) (mariano)
  - libsoav1_unitpbd: (v2) (Es la que estamos utilizando en pr_montecerrado y rudy_v2)
- Organización trbs
- Imagenes proyecto
- Crear un usuario nuevo (bd: sistema_pruebas)
- Utilidades
  - soamaster.hometech.red/connect
  - soavirtual.hometech.red
- A tener en en cuenta:

Implementacion soa

Modificaciones necesarias cada vez que se va a ejecutar el soa

1. En libsoav1/fuentes/
   1. En iniciarlibsoav1.c cambiar el puerto en la llamada a : FBFK_iniciar_sistemarpc_libsoav1(numeropuerto).

      Cada sistema tiene que tener un puerto diferente, empezamos por el puerto 457 y cada sistema se incrementará en 1, es decir:

      sistema1: puerto 457

      sistema2: puerto 458

      sistema3: puerto 459,....

      **Esto cambio conlleva a que la api, donde se realiza la llamada RPC, tiene que tener el mismo puerto del sisteme al que corresponda dicha api.
2. Las bases de datos donde esta la programación tiene que tener los permisos necesariso para que todo funcione correctamente, para que coja el nombre de las bd correpondientes y pueda crear las conexiones. Tener en cuenta que el usuario que utilice el soa para mysql tiene que tener conexion con la bd mysql, que es donde se crean las mayoria de las conexiones de mysq.
3. Modificar la tabla configuracionsistema en la bd: sistema_nombresistema, para modificar los tdatos con el usuario y clave de mysql que vamos a utilizar

4. Si fuese necesario, modificar en libsoav1/includes/iniciodeclaraciones.h para poner en los valores por defecto los que se necesite. (lo normal es que los rellene con la funcion del iniciar que coge los valores de la tabla configuracionsistema mencionada en el paso anterior).
5. En los th_socket cada sistema tiene que tener un puerto diferente, empezamos por el puerto 1504 y en cada sistema se incrementará en 1, es decir:

   sistema1: 1504

   sistema2: 1505

   ....
6.

Ejecutar el soa en un puesto de control:

1. Copiar las carpetas libsoa y kernelsoa donde queremos que se ejecute
2. Modificar en el Makefile de libsoav1 el nombre de la libreria que queremos que se genere

   ```
   LIBLECTURA=libsoanombresistema

   Ej: LIBLECTURA=libsoamonjas2
   ```

   libsoamonjas2 es el nombre de la libreria que queremos que se genere
3. Modificar en el Makefile de kernelsoa el nombre de la libreria que queremos importar

   ```
   LIBS= -lpthread -lmysqlclient -ldl -lsoanombresistema -lNBioBSP -lcurl

   Ej: LIBS= -lpthread -lmysqlclient -ldl -lsoamonjas2 -lNBioBSP -lcurl
   ```

   Donde -lsoamonjas2 es el nombre de la libreria a utilizar
4. Compilar libsoav1

   ```
   cd libsoav1/fuentes
   make clean
   make
   ```

5. Compilar kernelsoa

   ```
   cd kernelsoa/fuentes
   make clean
   make
   ```

6. Crearnos un fichero ej: kernelsoa.conf donde vamos a poner las rutas de los diferente sistemas (soa) que queremos lanzar en la siguiente ruta:

   ```
   cd /etc/ld.so.conf.d
   sudo nano kernelsoa.conf
   ```

   Dentro del fichero que nos creamos, escribir la ruta donde se encuentren los sistemas que queremos ejecutar, ejemplo:

   ```
   /home/soa/nombre_sistema1/libsoav1_unitp/eNBSP/bin
   /home/soa/nombre_sistema1/libsoav1_unitp/libreria
   /home/soa/nombre_sistema1/libloav1_unitp/libreria

   /home/soa/nombre_sistema2/libsoav1_unitp/eNBSP/bin
   /home/soa/nombre_sistema2/libsoav1_unitp/libreria
   /home/soa/nombre_sistema2/libloav1_unitp/libreria
   ```

7. Ejecutar comando para guardar los cambios.

```
sudo ldconfig
```

8. Ejecutar el proyecto, para ello
    1. Ponernos como root:

    ```
    sudo su
    ```

    2. Ubicarnos en la ruta donde esta el ejecutable

    ```
    cd kernelsoa/kernel
    ```

    3. Hacer el export de donde se van a encontrar las librerias

    ```
    export
    LD_LIBRARY_PATH=/home/soa/nombre_sistema/libsoav1_unitp/eNBSP/bin/:/home/soa/nombre_s
    istema/libsoav1_unitp/libreria/nombrelibreria.so

    Ejemplo:
    export
    LD_LIBRARY_PATH=/home/soa/sistema_monjas2/libsoav1_unitp/eNBSP/bin/:/home/soa/sistema
    _monjas2/libsoav1_unitp/libreria/libsoamonjas2.so
    ```

    4. Ejecutar

    ```
    ./kernelsoa nombresistema

    Ejemplo: ./kernelsoa monjas2
    ```

Errores SMS

Crear fichero status.c dentro de tools

Tendremos en config, dos variables umbral_entradas y umbral_salidas para definir el tiempo que queremos que se utilice para enviar o no un sms

Tendremos en declarations dos variables de tipo GList una para entradas y otra para salidas, guardando en cada posicion una estructura de datos, con la informacion necesaria.

Funcionamiento TRB Entradas:

Estructura GList Entradas:

```
status_trb_input: [
    status: true -> ok ; false -> fallo  (por ahora no se utiliza)
    counter: contador veces
    time: tiempo programado por rb
    message: texto a enviar en el SMS (contendrá ip, puerto,...)
    trb: tipo de trb (ed, ea, dali)
    sent: 0 -> no enviado, 1 -> enviado
    codigo_rb:
]
```

Formula Entradas

```
umbral_input: 150 s
tr: tiempo trb
v: (veces fallo)

if (tr * v) >= umbral_input
    Envia SMS
```

Cuando se inicia el soa y coge los valores de lo trbs

1. Llena la por cada uno, una posicion de la lista con los valores que corresponda de la estrutura status_trb_input, para ello llama a write_status_trb_input

lanzarErrorTrbEntradas(codigo_rb): Se la llama desde cada trb cuando da un error en un trb

1. Busca en la lista status_trb_input los valores correspondientes al codigo_rb que le llega por argumento
2. Pone status a false
3. Suma contador : (variable counter)
4. Aplica la formula :
    1. CUMPLE: mira si estado sent = 1
        1. SI: no hace NADA
        2. NO: envia SMS y pone sent a 1

borrarErrorTrbEntradas(codigo_rb): Se la llama desde cada trb cuando funciona ok

1. Pone status a true
2. Pone counter a 0
3. Pone sent a 0

Funcionamiento TRB Salidas:

Estructura GList Salidas:

```
status_trb_output: [
    status: true -> ok ; false -> fallo  (por ahora no se utiliza)
    time: timestamp
    message: texto a enviar en el SMS (contendrá ip, puerto,...)
    trb: tipo de trb (sd, sa, dali)
    codigo_rb:
]
```

Formula Salidas

```
umbral_output: 150 s
ta: tiempo timestamp_Actual
te: tiempo timestamp_error (la primera vez del error)

if (ta - te) >= umbral_output
    Envia SMS
```

Cuando se lanza una salida, se llena la primera vez , llama a write_status_trb_output

lanzarErrorTrbSalidas(codigo_rb): Se la llama cuando se lanza la salida y ha dado error

Bucar en la lista de las salidas si existe alguna posicion para dicho codigo_rb

1. NO:
    1. Llena la lista con los valores que corresponda

        2. Envia SMS
    2. SI:
        1. Pone status a false
        2. Cojo el tiempo actual y hago la formula
            1. CUMPLE: envia sms y actualiza te con ta (variable time de la lista)
            2. NO CUMPLE: Nada

borrarErrorTrbSalidas(codigo_rb): Se la llama cuando se lanza la salida y ha ido ok

1. Pone status a true

Funcionalidades

Tipos de Salidas:

las salidas desde tpse se llama a la funcion de la salida que esta cosida en el th de la tp, mandando el valor de la salida(seudoborna) que hayas programado desde el loa, recuerda que llama a la fk cuando cambia el valor de la tpse. Si el valor de la salida es x mira primero el valor anterior y manda el contrario, es decir si tiene un 1 manda un 0 y viceversa.

Si la salida es limitada durante x tiempo, por ejemplo lo que hacemos en las salidas cosidas al lector, si el valor programado es por ejemplo es 1 y el tiempo es 2 segundos, hace salida con 1 durante 2 segundos y luego hace salida con 0.

Si la salida es temporizada durante 2 segundos por ejemplo y el valor de la salida es 0, espera 2 segundos antes de hacer la salida.

CLIMA

El clima se activa manualmente desde el pulsador o desde la app en en boton de encender clima

Cuando se cambia estado desde la APP:

- SI estado = 0
    - Escribe inhibir = 1 en th_consigna
    - Hace salida 0 en valvula
- Si estado = 1
    - Escribe inhibir = 0 en th_consigna
    - Hace salida 0 en valvula

Escribe en tabla clima el estado

Cuando se pulsa en Pulsador:

- Obtenemos el estado de la bombilla (TPSE) para saber si hay que encender/apagar
- Si estado Bombilla = 1 APAGA CLIMA
    - Apaga bombilla (tpss)
    - Apaga valvula (tpss)
    - Inhibe consigna sonda (escribe 1 en inhibir en th_consigna)
- Si estado Bombilla = 0 ENCIENDE CLIMA
    - Enciende bombilla (tpss)
    - Enciende valvula (tpss)

TH_ pulsador: cosido en el th del pulsador (argumentos json)

1. tpss bombilla suelo radiante
2. tpse bombilla suelo radiante
3. valvulas (valvula suelo radiante): es un array con las valvulas necesarias
4. sondas (sonda temperatura): es un array con las sondas necesarias

- El numero de válvulas y de sondas debe ser el mismo.

```
Ej:
{"trb_tpss":"7","rb_tpss":"185","terminal_tpss":"13","trb_tpse":"0","rb_tpse":"184","terminal_t
pse":"15","valves":[{"trb": "23","rb": "211","terminal": "3"}],"probes":[{"trb": "1","rb":
"187","terminal": "0"}]}" donde :

{
    "trb_tpss":"7","rb_tpss":"185","terminal_tpss":"13", (tpss bombilla)
    "trb_tpse":"0","rb_tpse":"184","terminal_tpse":"15", (tpse bombilla)
    "valves":[{"trb": "23","rb": "211","terminal": "3"}], (array valvulas)
    "probes":[{"trb": "1","rb": "187","terminal": "0"}]
} (array sondas)
```

Th Consigna: cosido en las tpse de las sondas (argumentos json)

1. cod_consigna, cod_rango (id tabla 'variables')
2. tpss valvula suelo radiante: { trb_tpss, rb_tpss, terminal_tpss},
3. tpse valvula suelo radiante: { trb_tpse, rb_tpse, terminal_tpse},
4. id climate (id tabla 'variables')

```
Ej:
{"cod_consigna":"15","cod_rango":"16","trb_tpss":"23","rb_tpss":"209","terminal_tpss":"0","trb_
tpse":"21","rb_tpse":"210","terminal_tpse":"0","id_climate":"1"}
```

Cadena de Seguridad:

Th cadena_seguridad salida:

Esta cosida a una salida, si el valor de la salida es diferente al valor de la entrada, lo mira 3 veces, si siguen siendo diferentes ambos valores, escribe el valor de la entrada en la tp entrada para que se ejecuten los triggers.

Domo:

Tipos:

1. luces
2. clima
3. estor
4. iot
5. seguridad
6. accesos
7. persianas
8. piscina
9. cuadro
10. multimedia
11. bombas

Triggers domo:

Luces:

Cada vez que se hace un cambio en la tabla luces, mira si los valores que tiene la tabla luces, coincide con algun caso de la tabla operacion_luces para el codigo_luces correspondiente:

no: pone el estado de la operacion correspondiente a 0

si: pone el estado de la operacion correspondiente a 1

Clima:

Cada vez que se hace un cambio en la tabla clima, mira si los valores que tiene la tabla clima, coincide con algun caso de la tabla operacion_clima para el codigo_clima correspondiente:

no: pone el estado de la operacion correspondiente a 0

si: pone el estado de la operacion correspondiente a 1

Iot:

Cada vez que se hace un cambio en la tabla iot, mira si los valores que tiene la tabla iot, coincide con algun caso de la tabla operacion_iot para el codigo_iot correspondiente:

no: pone el estado de la operacion correspondiente a 0

si: pone el estado de la operacion correspondiente a 1

Operacion:

trigger: operacion_escena

Cada vez que se hace un cambio en la tabla operacion, mira a que escena pertenece y mira si hay alguna operacion de dicha escena con estado = 0

si: pone el estado de la escena correspondiente a 0

no: pone el estado de la escena correspondiente a 1

trigger: operacion_ambiente

Cada vez que se hace un cambio en la tabla operacion, mira a que ambiente pertenece y mira si hay alguna operacion de dicho ambiente con estado = 0

Mira si hay alguna escena pertenenciente a dicho ambiente con estado = 0

si no hay ninguna operacion con estado = 0 y no hay ninguna escena con estado = 0 pone el estado del ambiente correspondiente a 1

en caso contrario pone el estado del ambiente correspondiente a 1

Operacion:

Cada vez que se hace un cambio en la tabla operacion, mira a que escena pertenece y mira si hay alguna operacion de dicha escena con estado = 0

si: pone el estado de la escena correspondiente a 0

no: pone el estado de la escena correspondiente a 1

Medidores:

Cada vez que cambia la tp, activa el trigger_medidor que coge el valor del tcaso factor (factor) y del tcaso general y calcula la medicion (valor_ant - valor_tp) y escribe en los tcasos:

0. General: escribe en "valor" el valor de la tp y en "resultado" (valor*factor), si no existe crea caso
1. Anual: escribe en medicion_inicial, medicion_final, medicion, resultado los valores de (valor_ant, valor_tp, valor_tp - valor_ant(anual), (valor_tp - valor_ant)*factor) en el tipotcaso = 1
2. Mensual: Idem a anterior pero con mensual y tipotcaso = 2
3. Diario: Idem a anterior pero con diario y tipotcaso = 3
4. Hora: Idem a anterior pero con hora y tipotcaso = 4
5. Minuto: Idem a anterior pero con minuto y tipotcaso = 5

Historicos:

Cada vez que cambia la tp, activa el trigger que coge el valor de la tp a

Eventos

Medidores:

1. General
2. Anual
3. Mensual
4. Diario
5. Hora

Versiones de SOA:

libsoav1 (v1) (rudy)

Caracteristicas generales:

1. Una tabla (tp) por cada borna: nombre tp: _x
2. en huevon utiliza cortinillaspertenencia1
3. en karnaverde tabla: zonas

Es el soa que se esta ejecutando con el sistema fincarudy

libsoav1_unitp (version intermedia)

Caracteristicas generales:

1. Una tabla por comando, un caso por borna y tipotcaso: nombre tp: _x
2. Modificadas las tablas QSSISTEMA, clima, ... que tuviesen un tdato tp para añadir el tdato codigoborna
3. En huevon cambiada la tabla cortinillascomando por cortinillascomando, cortinillastrb, relacionrb, cortinillastpse, cortinillastpss
4. En karnaverde: en lugar de utilizar la tabla zonas se han creado las tablas: planta, estancias, zonas, luces, ambiente, estor, relacionluces, relacionambiente, relacionestor, tarea, tarea_rpc

Modificaciones:

1. Modificado en la función de FK_modofrio_consigna_libsoav1 y FK_modocalor_consigna_libsoav1
2. Creado en llaves_maestras/ el archivo socketupdate para cuando se escribe en la tp, lance el th socket , para que el proyecto se de cuenta de que ha habido un cambio para que vuelva a leer
3. Cosido en los trbs la llamada a th_socket
4. Creado en hilosth_libsoav1, la funcion para activar th_socket

5. trbs:
    1. creado raspberrysa_libsoav1, raspberryco2
    2. modificado raspberrysth32 para recoger temperatura y humedad
    3. modificado raspberrydali para incorporar intensidad_completa
    4. creado trb sivoia (pendiente probar, es por telnet)
    5. creado raspberryutils_libsoav1 para crear la funcion de FBFK_tramaida_dinamica_trbsraspberryutils_libsoav1 con los diferentes cases dependiendo del tipo de trb
    6. cosida la función anterior en todos los trbsraspberry

libsoav1_unitpbd: (v1) (mariano)

Caracteríticas generales: modificaciones sobre version libsoav1_unitp

1. Una tabla tp y th_tp por comando y trb y un caso por borna: nombre tp: rb_x nombreth: th_x
2. Una sola bd: sistema_x , (desparece huevon, ps, karnaverde, utilidades, calendario, ...)
3. Modificado el nombre de la tabla QSSISTEMA , para ponerla en minúsculas
4. Modificada la tabla estancias para añadir los tdatos: modelo, modo e imagen.

libsoav1_unitpbd: (v2) (Es la que estamos utilizando en pr_montecerrado y rudy_v2)

Caracteríticas generales: modificaciones sobre version libsoav1_unitpbd (v2)

1. Una tabla tp y th_tp por trb y un caso por borna y comando: nombre tp: "rb_"trb nombreth: th_x
    1. La estructura de la tp (id, fecha, valor, codigo,trigger)
    2. Modificadas las tablas:
        1. cortinillascomando por comando y modif estructura para quitar todos los tdatos que no se estan utilizando segun el listado de trbs que tenemos activo actualmente (trbs programados en rudy, hazard, mariano y montecerrado)
        2. cortinillastrb por trb
        3. relacionluces para modificar los tdatos tp_x y borna_x por trb_x y codigotp_x
        4. relacionambiente para modificar los tdatos tp_x y borna_x por trb_x y codigotp_x, y añadido un tdato trb_x por temp, otro para humedad y otro para co2, por si los valores de temperatura, humedad y co2 no vienen del mismo trb, es decir por ejemplo la temp y humedad vengan de un sth31 y el co2 del co2
        5. relacionestor para modificar los tdatos tp_x y borna_x por trb_x y codigotp_x
2. Los trigger son por trb, asi pueden ser generales
    1. a

Organización trbs

Habrá tres directorios:

trbs: raspberry, analizador, ..., cada trb llamara a funciones de la carpeta protocol , para calculo de crcs, envio de valores para generar la trama,..

protocol: contendrá los archivos de los protocolos de comunicacion (mbus, modbus) cada archivo preparará la trama, calculo de crc y llamara a las funciones de comunicacion para crear la conexion , envio de trama, ... Para ello prepararemos unas estructuras para cada tipo, para que le lleguen los datos necesarios.

comunication: contendrá los archivos de comucicacion (tcp, serie)

Imagenes proyecto

Copiar las imagenes en el proyecto que correponda en /var/ww/html/

```
cd /var/www/html/nombre_proyecto_imagenes
```

Crear carpeta fotos si no existe:

```
mkdir fotos
```

Copiar imagenes

```
scp imagen /var/www/html/nombre_proyecto_imagenes/
```

La carpeta y las imágenes para que funcionen tiene que tener usuario y grupo www-data

```
cd /var/www/html
sudo chown -R www-data:www-data nombre_proyecto_imagenes
```

En la tabla estancias en el tdato imagen, se tiene que guardar: 00000/nombre_imagen

```
Ej contenido tdato imagen: fotos/4994942c5440cbf2c329d4f127d150ab7504d051.jpg
```

Crear un usuario nuevo (bd: sistema_pruebas)

1. Crear caso en tabla proyecto: Alejandro Sanz
2. Crear caso en ubicacion: somosaguas, jarandilla de la vera
3. Crear caso en usuario: Alejandro (como pruebas), contraseña 5555 (se genera con php)
4. Duplicar el caso/s con codigoproyecto = 7 (pruebas)
5. Duplicar el caso/s con codigoproyecto = 8 (admin)

Utilidades

soamaster.hometech.red/connect

Para ver el estado de una placa, se leen los valores de las entradas, salidas...

Hay que introducir ip y puerto de la placa

Ej: ip -> 88.12.32.148 puerto -> 53134 (placa de las monjas donde estamos probando pr_montecerrado)

soavirtual.hometech.red

Para poder probar apis, para poder utilizarla la placa y el soa tiene que esta utilizando actualmente la ip: 89.7.222.195 y el puerto -> 39185 (actualmente el sistema que esta con esta ip y puerto es pr_mariano para hacer las pruebas)

A tener en en cuenta:

1. Las funciones que inician los trbs estan en libsoav1/fuentes en iniciarlibsoav1. Si queremos desactivar un trb, modificar el valor del tdato activo en la tabla ps.iniciarhilostrb, para que no lo lea al iniciar el soa. Para que el soa se de cuenta de este cambio hay que reinicar el soa.
2. En la tabla tareas_rpc los valores escritos en argumentos tienen que ir entre comillas por que el soa necesita que le llegue como un string.
3. Instalacion libjson-c

   lib JSON-c: Ver documentacion https://github.com/json-c/json-c

```
$apt-get install libjson-c-dev (json)
$apt-get install cmake

git clone https://github.com/json-c/json-c.git
cd json-c
cmake ../json-c
make
make test
sudo make install

copiar las librerias que se han generado en /usr/lib/
$sudo cp libjson-c.so* /usr/lib/
```

Añadir al Makefile de libsoa

```
CFLAGS += $(shell pkg-config --cflags json-c)
LDFLAGS += $(shell pkg-config --libs json-c)

CC= gcc -fPIC -Wall -pthread -g -c $(ABYSS_CFLAGS) $(CFLAGS) `pkg-config --cflags glib-2.0` `pkg-config --libs glib-2.0` $(RUTAINCLUDES) $(LLIBS) $(ABYSS_LIBS) $(LDFLAGS)
```

Añadir al Makefile de kernelsoa

```
donde las fuentes: LLECTURA= $(shell pkg-config --cflags json-c)
donde las librerias : LIBS= $(shell pkg-config --libs json-c)
```

4. Para utilizar las funciones de json, ver la documentacion de https://programmerclick.com/article/74091275562/

Iniciar soa automaticamente

Crear un archivo kernelsoa_x por cada soa que queramos iniciar, con el contenido siguiente

si el soa a implementar es la ultima version:

```
#!/bin/sh

### BEGIN INIT INFO
# Provides:           kernelsoa_pr_montecerrado
# Required-Start:     $local_fs $network $syslog mysql
# Required-Stop:      $local_fs $network $syslog
# Default-Start:      2 3 4 5
# Default-Stop:       0 1 6
# Short-Description:  kernelsoa_pr_montecerrado
# Description:        kernelsoa_pr_montecerrado
### END INIT INFO

NAME="kernelsoa_pr_montecerrado"
PATH="/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin"
APPDIR="/home/soa/sistema_pr_montecerrado/kernelsoa_unitpbd_v2/kernel"
APPBIN="/home/soa/sistema_pr_montecerrado/kernelsoa_unitpbd_v2/kernel/kernelsoa"
APPARGS="sistema_pr_montecerrado"
USER="root"
GROUP="root"

# pid file for the daemon
PIDFILE="/var/run/kernelsoa_pr_montecerrado.pid"

# Include functions
set -e
. /lib/lsb/init-functions
```

```
start() {
   printf "Starting '$NAME'... "
   start-stop-daemon --start --chuid "$USER:$GROUP" --background --chdir "$APPDIR" --exec
"$APPBIN" -- $APPARGS || true
   printf "done\n"
}


stop() {
  printf "Stopping '$NAME'... "
  kill -9 `cat $PIDFILE`
  printf "done\n"
}

status() {
  # Check the status of the process.
  if [ -e $PIDFILE ]; then
    status_of_proc -p $PIDFILE $APPBIN "$NAME process" && exit 0 || exit $?
  else
    log_daemon_msg "$NAME Process is not running"
    log_end_msg 0
  fi
}

case "$1" in
  start)
    start
    ;;
  stop)
    stop
    ;;
  restart)
    stop
    start
    ;;
  status)
    status
    ;;
  *)
    echo "Usage: $NAME {start|stop|restart|status}" >&2
    exit 1
    ;;
esac

exit 0
```

Si el soa a implementar es una version antigua

```
#!/bin/sh

### BEGIN INIT INFO
# Provides:           kernelsoa_hazard
# Required-Start:     $local_fs $network $syslog mysql
# Required-Stop:      $local_fs $network $syslog
# Default-Start:      2 3 4 5
# Default-Stop:       0 1 6
# Short-Description:  kernelsoa_hazard
# Description:        kernelsoa_hazard
### END INIT INFO

NAME="kernelsoa_hazard"
PATH="/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin"
#APPDIR="/home/soa/sistema_hazard/kernelsoa_unitp/kernel"
APPDIR="/"
APPBIN="/home/soa/sistema_hazard/kernelsoa_unitp/kernel/kernelsoa"
APPARGS="sistema_hazard"
USER="root"
```

```
GROUP="root"

# Include functions
set -e
. /lib/lsb/init-functions

start() {
   printf "Starting '$NAME'... "
   start-stop-daemon --start --chuid "$USER:$GROUP" --background --chdir "$APPDIR" --exec
"$APPBIN" -- $APPARGS || true
   printf "done\n"
}


stop() {
  printf "Stopping '$NAME'... "
  kill $(pgrep $NAME)
  printf "done\n"
}

status() {
  status_of_proc $NAME && exit 0 || exit $?
}

case "$1" in
  start)
    start
    ;;
  stop)
    stop
    ;;
  restart)
    stop
    start
    ;;
  status)
    status
    ;;
  *)
    echo "Usage: $NAME {start|stop|restart|status}" >&2
    exit 1
    ;;
esac

exit 0
```

Donde los parametros a modificar son:

Necesitas tener un fichero de estos por cada soa que tengas

Provides, Short-Description y Description por ‘kernelsoa_mariano’

NAME : ejemplo "kernelsoa_mariano"

APPBIN: ruta donde se encuentra el ejecutable de kernelsoa que queremos iniciar

APPARGS: nombre del sistema que en este caso seria mariano

Pasar dicho archivo/s a a `/etc/init.d`

Y ejecutar lo siguiente por cada archivo

```
sudo update-rc.d kernelsoa-montecerrado defaults
sudo update-rc.d kernelsoa-montecerrado enable
```

Si hay que hacer algun cambio

```
sudo update-rc.d kernelsoa-montecerrado remove
sudo update-rc.d kernelsoa-montecerrado defaults
sudo update-rc.d kernelsoa-montecerrado enable
```

Para arrancar el soa

```
sudo /etc/init.d/kernelsoa_pr_montecerrado stop
sudo /etc/init.d/kernelsoa_pr_montecerrado start
```

Arrancar soa para control de errores, consumos de memoria

```
valgrind --leak-check=full --show-leak-kinds=all -v ./kernelsoa ardal2
```

Una vez ejecutado, pulsar Ctrol+C y leer los mensajes de error, hay que tener en cuenta sobretodo los que vienen identificados con definetily

Ejemplo de mensajes de error:

```
==451197==
==451197== 4,731 (2,280 direct, 2,451 indirect) bytes in 57 blocks are definitely lost in loss
record 1,317 of 1,348
==451197==    at 0x483B7F3: malloc (in /usr/lib/x86_64-linux-gnu/valgrind/vgpreload_memcheck-
amd64-linux.so)
==451197==    by 0x49A853B: delete_input_trb_error (status.c:125)
==451197==    by 0x49BC3C6: motor_port_active_raspberryed (raspberryed.c:44)
==451197==    by 0x4C7C608: start_thread (pthread_create.c:477)
==451197==    by 0x4B2E292: clone (clone.S:95)

==451197== LEAK SUMMARY:
==451197==    definitely lost: 16,861 bytes in 422 blocks
==451197==    indirectly lost: 9,442 bytes in 210 blocks
==451197==      possibly lost: 1,699,656 bytes in 1,472 blocks
==451197==    still reachable: 446,430 bytes in 6,533 blocks
==451197==         suppressed: 0 bytes in 0 blocks
==451197==
==451197== ERROR SUMMARY: 153 errors from 153 contexts (suppressed: 0 from 0)
```

en delete_input_trb_error del archivo status.c en la linea 125 estaria una pérdida de memoria